

Task

Implement and evaluate a Logistic Regression model using Scikit-learn.

Load data

Subtask:

Load the dataset into a pandas DataFrame.

Import the pandas library, load the dataset from the specified file path into a DataFrame, and display the first 5 rows to inspect the data as requested by the instructions.

```
from sklearn.datasets import load_breast_cancer
import pandas as pd

# Load the breast cancer dataset
breast_cancer = load_breast_cancer()

# Create a DataFrame
df = pd.DataFrame(data=breast_cancer.data,
                  columns=breast_cancer.feature_names)
df['target'] = breast_cancer.target

display(df.head())
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst texture	worst perimeter	worst area	worst smoothness	worst compactness	worst concavity	worst concave points	worst symmetry
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	17.33	184.60	2019.0	0.1622	0.6656	0.7119	0.2654	0.4601
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	23.41	158.80	1956.0	0.1238	0.1866	0.2416	0.1860	0.2750
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...	25.53	152.50	1709.0	0.1444	0.4245	0.4504	0.2430	0.3613
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...	26.50	98.87	567.7	0.2098	0.8663	0.6869	0.2575	0.6638
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...	16.67	152.20	1575.0	0.1374	0.2050	0.4000	0.1625	0.2364

5 rows x 31 columns

For Logistic Regression, we need to separate the features (X) from the target variable (y). The target variable is the 'target' column, and the features are all other columns.

```
# Separate features (X) and target (y)
X = df.drop('target', axis=1)
y = df['target']
```

```
display(X.head())
display(y.head())
```

target	
0	0
1	0
2	0
3	0
4	0

dtype: int64

Split the data into training and testing sets. This is essential for evaluating the model's performance on unseen data. We'll use a test size of 20% and a random state for reproducibility.

```
from sklearn.model_selection import train_test_split

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

```
X_train shape: (455, 30)
X_test shape: (114, 30)
y_train shape: (455,)
y_test shape: (114,)
```

Train the model

Subtask:

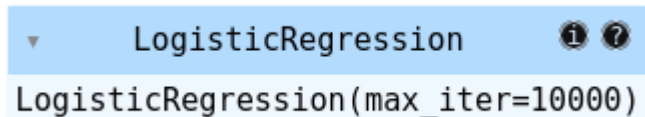
Train a Logistic Regression model on the training data.

Import the Logistic Regression model from scikit-learn and initialize it. Then, train the model using the training data (**X_train** and **y_train**).

```
from sklearn.linear_model import LogisticRegression

# Initialize the Logistic Regression model
model = LogisticRegression(max_iter=10000) # Increased max_iter for
convergence

# Train the model
model.fit(X_train, y_train)
```

A screenshot of a Jupyter Notebook cell. The cell has a blue header bar with the text "LogisticRegression" and two circular icons (one with an 'i' and one with a '?'). Below the header, the code "LogisticRegression(max_iter=10000)" is displayed in a light blue background.

```
LogisticRegression(max_iter=10000)
```

Evaluate the model

Subtask:

Evaluate the model's performance on the test data using appropriate metrics.

Make predictions on the test set using the trained model.

```
# Make predictions on the test set
y_pred = model.predict(X_test)
```

Evaluate the model's performance using common classification metrics: accuracy, precision, recall, F1-score, and the confusion matrix. These metrics provide a comprehensive view of how well the model is performing.

```
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, confusion_matrix
```

```
# Calculate evaluation metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
```

```
# Display the metrics
print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-score: {f1:.4f}")
```

```
print("Confusion Matrix:")  
display(conf_matrix)
```

Accuracy: 0.9561

Precision: 0.9459

Recall: 0.9859

F1-score: 0.9655

Confusion Matrix:

```
array([[39, 4],  
       [ 1, 70]])
```